

University of Texas at Dallas

CS/SE 2340.003

Computer Architecture

Spring 2026

Term Project

Building a MIPS Black Board Game

Project Report

By - Sanskriti Tiwari

Professor - Mrs. Alice Wang

Date - 3 April 2026

Contents

Serial number	Description	Page number
a)	Program Description	2
b)	Challenges and Solutions	3
c)	Learning Outcomes	4
d)	Algorithms and Techniques	4
e)	Extra Credit Description	5

Program Description

This extensive CS/SE 2340 Computer Architecture term project primarily focused on using the MIPS concepts learnt in the first half of the semester. These topics like loops, printing using syscalls, shifters, jump, etc. were put to practical use with a complex semester-long project. The project is about MIPS coding a the black hole game play with the computer in MARS.

The black hole is just taking turns with your game partner and noting down numbers in a sequential, but in random positions. The board is shaped like a pyramid and a longer succeeding row is added every time a new player joins the game. Once the players are done filling out their numbers, an empty hole is left, i.e. called the black hole. The numbers surrounding the black hole are counted separately for each player. The one with the lowest score win's.

The objective of the project is similar, but it's a player vs computer. The player and computer have to enter 1 to 10 numbers each in sequential, but random positions on the 21 tiles pyramid. This obviously leaves a hole in the end, which is used to calculate the final score for each, and determine a winner. To make the project more interesting I added music and visualized it using the MARS Bitmap tool.

Many files were required to integrate in the program. Their short description:

- board.asm - This file store's numbers and shuffle's the positions of the numbers to be stored in the input/output section pyramid and gets the adjacent entry's address
- computer.asm - This file generates the seed for the computer's number to be placed in random positions and prints them.
- display.asm - This file not only prints the board's format for the input/output window, but also for the Bitmap display (colours, draw, pixels)
- input.asm - This file shows the opening message, rules, and asks the user if they want to play again and enter their choice.
- main.asm - This file plays the opening music and prints the welcome screen. It also calculates the player and computer shuffle index, update's ASCII board. Finds the black hole position.
- music.asm - This file plays music for the intro, main, a happy one if the player wins, and a depressing one if the player loses in the end.
- scoring.asm - This file calculates the score by scanning the black hole and prints the final board, score, winner, and closing message.
- Syscalls.asm - Definitions of various useful system calls for MIPS assembly.

Challenges and Solutions

- **Challenge 1** - Finding the right procedural and position randomization logic was a big challenge. It's complex and overwhelmed me in a lot of places.
- **Solution 1** - Did multiple sketches on paper for the random calculation logic, and to get how the game's components should follow. The TA demo video helped me visualize the output, and build towards it. Also used Claude Sonnet 4.6 for the concepts and syntaxes.
- **Challenge 2** - Since it was my first time using the Bitmap tool more effectively. My MARS application would often freeze when not disconnected properly after testing the game.
- **Solution 2** - Eventually found out the root cause and was more cautious while testing. Also mentioned in the user manual for any other user's usage.
- **Challenge 3** - Similar to the randomisation logic, I had issues understanding and using the bitmap display.
- **Solution 3** - Had to watch YouTube videos for concepts and simpler demons, and used Claude Sonnet 4.6 to understand how I can format the bitmap display neatly in the project.

Learning outcomes

Through his project I learnt how even the most basic concepts of a language can build complex and fun projects. Specifically for this term project, I learnt the practical approach for a language that was introduced just months ago. Building this project also gave me the confidence to work practically with MIPS more effectively. I also applied the ethical use of generative AI in leveraging my projects to its full potential, while also aiding in learning the conceptual approach of topics. I would hopefully scale this project with the new concepts we learnt, so that it's easier to work with.

Algorithms and Techniques

- Like most full stack projects, I had to divide this into 2 parts: backend and frontend.
- Backend had the most work and included: computer seed, randomization of positions, calculation of scores with reference to the black hole, determining the winner, music, etc.
- Frontend majorly include the work related to the Bitmap display and formatting of the input/output window using the syscalls.asm file.
- The general working is as follows:
Seed print's welcome message and rules > play's intro music > init_board clears the board and shuffles the positions for a new game > Players and Computers numbers are added in sequential order from 1-10 > ASCII board is refreshed after each round > reflects on the Bitmap display > Remaining tile is the black hole > Reflects on the Bitmap display for the last time > Black hole position is printed > Score is calculated > The lowest score wins > happy/sad music is played accordingly > Takes user input to replay or exit.

Extra Credit Description

1. Music.asm (Added music to the board game)

- The file plays music for the intro, main, and end. The ending music depends on the player's victory. If they win, it's a happy music and a depressing one if they lose. The main music is Für Elise–style that is of short length.
- Instrument used => Grand piano = 0
- Volume => 90 (on the scale 1-127)
- Gave appropriate spacing to not override the music.

2. Display.asm (Added Bitmap Display for board game visualization)

- This file not only prints the board's format for the input/output window, but also for the Bitmap display (colours, draw, pixels)
- Player = blue block
- Computer = red block
- Hole = green block
- The border = white
- Gaps are left by creating the white pixelate borders. These are then filled eventually by the player's position.